

Architecture Produit Vivante

Mobile-first · Fondations micro-frontends · Leo + Onboarding

APV

Ce document répond à 3 décisions confirmées :

Mobile d'abord

Le web suit. Jamais ne précède.

Micro-frontends Phase 2

On pose les fondations maintenant.

Leo + Onboarding intégrés

Un seul système d'adaptation, pas deux.

7

Chapitres

12

Lois immuables

48

Tâches

2

Horizons (maintenant / Phase 2)

SOMMAIRE

01	Le Manifeste v2	3 décisions confirmées · Ce qui change · Ce qui reste
02	Mobile-first — La règle d'or	Mobile comme source de vérité · Le web comme projection
03	Fondations micro-frontends	Ce qu'on pose maintenant · Ce qui attend Phase 2
04	Leo + Onboarding — Un seul système	L'assistant qui enseigne · Le parcours qui guide · L'intégration
05	Home Screen — Spec finale	Mobile + Back-office web · États · Rôles · Composants
06	Les 48 tâches	4 sprints · Horizon maintenant vs Phase 2 clairement séparés
07	Les 12 Lois	Immuables · La source de vérité absolue

CH.01

LE MANIFESTE v2

3 décisions confirmées · Ce qui change · Ce qui reste

01

Ce document est la v2 du manifeste APV. Il intègre 3 décisions précises : (1) Mobile-first avec le web comme back-office managers, (2) Micro-frontends en Phase 2 avec les fondations posées maintenant, (3) Leo et l'onboarding progressif comme un seul système intégré. Ces 3 décisions changent l'architecture, pas la philosophie.

Les 3 décisions et leurs conséquences exactes

Décision	Ce que ça signifie MAINTENANT	Ce que ça signifie en PHASE 2	Ce qui NE change PAS
Mobile-first	Toute spec commence par l'écran mobile. Le web suit sans devancer. L'employé simple n'utilise que le mobile. Le web = back-office manager uniquement.	Si les micro-frontends arrivent, ils arrivent d'abord sur mobile (Dart plugins), le web suit avec Module Federation.	La philosophie épurée, Leo, le chat, les Quick Actions chips — tout ça reste.
Micro-frontends Phase 2	On crée les packages Flutter séparés (leopardo_core, module_rh, module_finance...). Les contrats de modules sont écrits. La structure de fichiers est celle d'un futur micro-frontend.	On active le lazy loading Dart deferred. On active Webpack Module Federation côté web. Chaque module peut être déployé indépendamment avec Shorebird.	Le code des modules actuels (Finance, Caméras, RH) ne change pas. Seule la façon de les packager évolue.
Leo + Onboarding intégrés	Un seul service Laravel (LeoContextService) gère à la fois le chat IA et le parcours d'apprentissage. Leo sait à quel stade est l'utilisateur. L'onboarding déclenche Leo.	Leo peut proposer des suggestions proactives basées sur l'historique d'usage. L'onboarding devient prédictif.	Le scope de Leo reste strictement Leopardo RH. Les guard-rails ne changent pas.

La phrase qui résume tout

"L'app s'ouvre sur une conversation, se déploie module par module, et apprend à mesure qu'on l'utilise — en partant toujours du téléphone."

Les horizons de l'architecture



Les cases violettes sont le scope de ce document. Les cases bleues/ambre/teal sont des destinations vers lesquelles on navigue sans se précipiter.

CH.02

MOBILE-FIRST — LA RÈGLE D'OR

Mobile comme source de vérité · Web comme back-office managers

02

Mobile-first ne signifie pas 'on fait d'abord le mobile puis le web après'. Ça signifie que la source de vérité UX est toujours l'écran mobile. Le web adapte les décisions prises sur mobile — jamais l'inverse. Et le web est exclusivement un back-office pour managers et gestionnaires. L'employé simple n'a pas besoin du web.

Le diagramme — mobile comme source, web comme projection



Ce que ça change dans le processus de conception

Étape	AVANT (sans règle mobile-first)	APRÈS (mobile-first)
1. Spec UX	On écrit la spec sans penser à la surface	On wireframe l'écran mobile en premier. Toujours.
2. Layout	On conçoit un layout 'responsif' qui s'adapte	Mobile = contrainte principale. Web = adaptation.
3. Navigation	On pense en termes de menu et sidebar	On pense en termes de BottomNav et swipe. Le web adapte.
4. Données affichées	On affiche tout ce qu'on peut sur grand écran	On affiche uniquement ce qu'on peut lire sur un 375px. Le web peut en montrer plus.
5. Actions	On conçoit les actions pour la souris et le clavier	On conçoit les actions pour le pouce. Le web ajoute raccourcis clavier.
6. Revue design	On review sur desktop dans le navigateur	On review sur simulateur iPhone SE (375px). Toujours.

Les utilisateurs et leurs surfaces

Persona	Surface principale	Surface secondaire	Jamais sur
Manager Principal	Mobile (toujours avec lui)	Web back-office (bureau, reporting)	—

Persona	Surface principale	Surface secondaire	Jamais sur
Gestionnaire Financier	Mobile (saisie terrain)	Web back-office (tableaux, exports)	—
Manager RH	Mobile (déplacements fréquents)	Web back-office (listes, corrections)	—
Employé simple	Mobile uniquement	—	Web — aucun accès nécessaire
Superviseur	Mobile principalement	Web parfois (reporting équipe)	—
Super Admin (éditeur SaaS)	Web uniquement	—	Mobile (pas de cas d'usage)

Conséquence directe : le web n'a pas besoin de supporter tous les cas d'usage. Il n'a pas besoin d'être responsive pour les petits écrans (l'employé utilise l'app mobile). Il est conçu pour une largeur minimale de 1024px, layout en deux colonnes, avec sidebar. C'est un outil professionnel de bureau, pas une page marketing.

Règles mobile-first dans le code

Règles mobile-first — code et process

```
// ■ CORRECT — on spec le mobile, le web adapte
// 1. Flutter (mobile) : touch target 44dp, BottomNav, swipe
// 2. Blade (web) : même feature, layout 2-colonnes, hover states

// ■ CORRECT — la spec commence par :
// 'Sur mobile, le manager voit X en tapant sur Y'
// 'Sur web, le manager retrouve X dans la colonne principale en cliquant Y'

// ■ CORRECT — review obligatoire sur iPhone SE simulateur
flutter run -d 'iPhone SE (3rd generation)' // Device de référence pour les reviews

// ■ INTERDIT — concevoir le web en premier puis adapter mobile
// ■ INTERDIT — une feature web sans équivalent mobile (sauf back-office pur)
// ■ INTERDIT — review design uniquement dans un navigateur desktop
// ■ INTERDIT — layout mobile 'responsive' basé sur un design web
```

Ce que le web back-office apporte en plus (légitimement)

- **Tableaux denses** : le web peut afficher 10 colonnes dans un tableau d'employés. Le mobile en affiche 3. Ce n'est pas une limitation du mobile — c'est un choix de priorisation de l'information.
- **Exports et rapports avancés** : télécharger un rapport PDF de 50 pages se fait confortablement sur web. On ne supprime pas la feature sur mobile, on la simplifie (bouton 'Recevoir le rapport par email').
- **Édition complexe** : modifier un bon de commande avec 20 lignes est plus confortable avec un clavier. Sur mobile, on crée et on consulte. Sur web, on édite en profondeur.
- **Raccourcis clavier (■K, ■N)** : uniquement sur web. Sur mobile, ce sont des gestes (swipe, long press).

→ **Multi-fenêtres** : un manager peut avoir le dashboard Finance dans un onglet et la liste des employés dans un autre. Cas d'usage impossible sur mobile — et ce n'est pas grave.

CH.03

FONDTIONS MICRO-FRONTENDS

Ce qu'on pose maintenant · Ce qui attend Phase 2

03

Micro-frontends Phase 2 ne signifie pas qu'on attend pour commencer. Ça signifie qu'on pose MAINTENANT les fondations qui rendront la bascule triviale en Phase 2 — et qu'on n'active pas la complexité opérationnelle prématurément (builds séparés, CDN modules, versioning indépendant) avant d'en avoir besoin.

Ce qu'on fait MAINTENANT (fondations)

Action	Effort	Bénéfice immédiat	Bénéfice Phase 2
Créer le package <code>leopardo_core</code>	2j	Code core séparé du code modules	Déployable indépendamment
Créer <code>module_rh</code> , <code>module_finance</code> , <code>module_cameras</code>	1j chacun	Isolation du code par domaine	Buildable + testable séparément
Écrire <code>LeopardoModule</code> abstract class	0.5j	Contrat standard pour tout nouveau module	Interface stable pour micro-frontend
Route groups Laravel isolés (routes/modules/)	0.5j	Organisation claire du code API	Déployable comme microservice séparé si besoin
Tests d'isolation packages (CI job)	0.5j	Garantie qu'aucun module ne 'fuite' dans un autre	Prérequis pour déploiement indépendant
JSONB metadata sur tables clés	0.25j	Extensibilité sans migration	Contrat de données stable entre modules
Feature flags system	0.5j	Activation/désactivation modules par plan	Toggle modules sans déploiement

Ce qu'on N'active PAS maintenant (Phase 2)

Feature	Pourquoi on attend	Déclencheur Phase 2
Dart deferred imports (lazy loading)	Complexité de build + edge cases iOS. Pas de gain mesurable avec < 5 modules.	Quand l'app dépasse 5 modules actifs ou quand l'APK > 100MB.
Webpack Module Federation (web)	Requiert une infrastructure CDN dédiée. La valeur n'est pas là sans > 3 équipes.	Quand une équipe externe (partenaire, client Enterprise) veut développer un module.
Shorebird code push	Requiert un compte payant et un process de release supplémentaire.	Dès qu'un bug critique en production nécessite un hotfix sans passer par les stores.
Builds CI séparés par module	Overkill avec 1 développeur. Le temps de build total reste < 10 min.	Quand le temps de build dépasse 15 min ou quand l'équipe dépasse 3 personnes.

Feature	Pourquoi on attend	Déclencheur Phase 2
Versioning indépendant des modules	Nécessite une gestion de compatibilité entre versions de modules.	Quand un module a un cycle de release différent du core (ex: module Finance mis à jour toutes les 2 semaines).

La structure cible — ce qu'on crée maintenant

Structure cible – créée maintenant, activable en Phase 2

```
leopardo_rh/
├── packages/
│   ├── leopardo_core/
│   │   ├── lib/core/theme/
│   │   ├── lib/core/navigation/
│   │   ├── lib/core/leo/
│   │   ├── lib/core/onboarding/
│   │   └── lib/core/widgets/
│   ├── module_rh/
│   │   ├── lib/module.dart
│   │   └── module_finance/
│   │       ├── lib/module.dart
│   │       └── module_cameras/
│   │           └── lib/module.dart
│   ├── apps/leopardo_app/
│   │   ├── pubspec.yaml
│   │   └──
│   └── api/routes/modules/
│       ├── rh.php
│       ├── finance.php
│       └── cameras.php
└── # Packages Flutter isolés
    # CRÉER MAINTENANT
    # AppColors, AppTheme, Typography
    # ModuleRegistry, AppShell, GoRouter base
    # ChatInput, ChatMessage, LeoBloc
    # LearningStage, OnboardingChecklist
    # LeopardoBadge, EmptyState, AlertBanner
    # CRÉER MAINTENANT
    # RhModule implements LeopardoModule
    # CRÉER MAINTENANT
    # FinanceModule implements LeopardoModule
    # CRÉER MAINTENANT
    # CameraModule implements LeopardoModule
    # App principale
    # path: ../packages/leopardo_core
    # path: ../packages/module_rh (si plan l'autorise)
    # path: ../packages/module_finance (si plan)
    # Laravel – CRÉER MAINTENANT
    # Préfixe /api/v1/rh/
    # Préfixe /api/v1/finance/
    # Préfixe /api/v1/cameras/
```

Le contrat module — interface standard

```
LeopardoModule — le contrat que chaque module doit implémenter

// packages/leopardo_core/lib/core/navigation/leopardo_module.dart
abstract class LeopardoModule {
  String get id;           // 'finance' | 'cameras' | 'rh'
  String get label;        // 'Finance' | 'Caméras' | 'RH'
  IconData get icon;       // Icons.account_balance_outlined
  Color get domainColor;   // AppColors.finance
  String get minPlan;      // 'starter' | 'business' | 'enterprise'

  List<RouteBase> get routes;           // Routes GoRouter du module
  AppTab? get bottomNavTab;            // Onglet BottomNav (null si pas d'onglet)
  Future<List<HomeAlert>> getAlerts(User u); // Alertes pour la Home
  List<String> get leoTopics;          // Sujets que Leo peut traiter pour ce module
  ModuleChip get homeChip;             // Quick Action chip pour la Home
  List<OnboardingStep> get onboardingSteps; // Étapes d'onboarding du module
}

// Enregistrement dans main.dart
ModuleRegistry.registerAll([
  RhModule(), // Toujours présent
  if (plan.hasModule('finance')) FinanceModule(),
  if (plan.hasModule('cameras')) CameraModule(),
  // Phase 2 : if (plan.hasModule('alarms')) AlarmModule(),
]);
```

CH.04

LEO + ONBOARDING — UN SEUL SYSTÈME

L'assistant qui enseigne · Le parcours qui guide · L'intégration complète

04

La décision clé : Leo et l'onboarding progressif ne sont pas deux features séparées. Ils partagent le même service côté serveur (LeoContextService), la même donnée (learning_stage dans employees.metadata), et le même canal de communication avec l'utilisateur (la Home conversationnelle). Leo sait où en est l'utilisateur. L'onboarding déclenche Leo au bon moment.

Le système intégré — vue d'ensemble

Trois composants forment le système :

Composant	Responsabilité	Stockage	Déclenché par
LearningStageService (Laravel)	Calcule le stade courant (J0 → Expert) depuis le compteur d'actions. Incrément après chaque action réussie. Retourné dans /api/v1/auth/me.	employees.metadata.app_actions_count employees.metadata.learning_stage employees.metadata.first_screen_screens	ActionTrackerMiddleware après chaque POST/PATCH/DELETE réussie
LeoContextService (Laravel)	Construit le contexte complet injecté dans le prompt IA : rôle, plan, alertes actives, stade d'apprentissage, écrans vus, modules actifs. Sélectionne le mode Leo approprié.	Lecture depuis employees.metadata + alertes temps réel depuis les tables modules	POST /api/v1/ai/chat et GET /api/v1/ai/morning-brief et GET /api/v1/ai/suggestions
HomeBloc (Flutter)	Orchestre l'affichage : alertes + messages Leo + stade appris + chips. Décide quand envoyer un message Leo automatique (premier usage, action significative, morning brief).	State en mémoire (BLoC). Persistance des messages via cache Hive local.	LoadHome event (ouverture app) + SendMessage event (frappe utilisateur)

Les 5 stades d'apprentissage — définitions précises

Stade	Condition	Leo se comporte...	L'interface révèle...
■ Découverte	actions_count < 10 OU J0-J2	Mode guide actif. Envoie un message de bienvenue sur la Home. Commentaire contextuel au premier accès de chaque écran. Suggestions pré-écrites dans la barre chat.	3 chips max. Formulaires en mode simplifié (champs essentiels seulement). 0 filtre avancé.
■ Routine	10 ≤ actions < 50 J3-J14	Mode réponse. Leo répond aux questions sans initier. Morning brief proactif si alertes actives. Plus de message de bienvenue.	+ filtres basiques. + HistoryScreen. + badge progression profil. Formulaires complets.
■ Maîtrise	50 ≤ actions < 200 J15-J30	Mode expert. Réponses courtes et précises. Propose des raccourcis : "Vous pouvez aussi utiliser ■N pour créer rapidement."	+ Exports CSV. + Raccourcis web ■K/■N. + Bulk actions. + Config avancée modules.

Stade	Condition	Leo se comporte...	L'interface révèle...
■ Expert	actions ≥ 200 J31+	Mode ressource. Leo répond aux questions pointues. Peut suggérer des optimisations de workflow. 'Je remarque que vous faites souvent X — voulez-vous un raccourci ?'	Tout ce que le plan autorise. Personnalisation chips (ordre). Dashboard analytique si Enterprise.
■ Ambassadeur	actions ≥ 500 J60+	Mode pair. Leo peut confirmer les best practices. 'Vous avez raison, c'est la meilleure façon de procéder pour...'	Accès beta features en avant-première. Badge discret dans le profil. Lien vers programme de parrainage (Phase 2).

Les 4 modes Leo — spec complète

Mode	Déclenchement	Contenu du message	Durée affichage
Guide Premier Accès	HomeBloc détecte que l'écran apparaît dans la liste pour la première fois (metadata.first_seen_screens ne contient pas l'id de l'écran courant)	Phrase courte expliquant l'objectif de l'écran + l'action principale. Ex : 'Cet écran vous permet de voir qui est présent aujourd'hui. Tapez sur le bouton vert pour pointer votre arrivée.'	Affiché 1 fois par écran. Ne revient jamais.
Morning Brief	Première ouverture de l'app du jour ET (alertes actives OU actions importantes en attente)	Liste structurée : 'Bonjour [Prénom]. Aujourd'hui : [N] factures overdue · [N] dépenses à valider · [N] employés absents.' Avec lien vers chaque entité.	Affiché une fois par jour à la première ouverture.
Réponse Contextuelle	L'utilisateur envoie un message dans la barre chat	Réponse via API externe (GPT-4o / Claude) avec system prompt complet incluant le contexte utilisateur et son stade d'apprentissage.	Jusqu'à la prochaine interaction.
Confirmation d'action	L'utilisateur réalise une action majeure (envoie une facture, approuve une dépense, configure une caméra)	Message court confirmant et donnant le contexte : 'Facture INV-2026-0001 envoyée à client@email.com. Elle sera en statut Envoyée jusqu'au paiement. Vous pouvez la relancer dans 7 jours si pas de réponse.'	S'affiche et reste jusqu'au prochain message.

LeoContextService — l'implémentation

```
LeoContextService.php — Le service central
// app/Services/LeoContextService.php
class LeoContextService {
    public function buildPrompt(User $user, string $mode): array {
        $stage = $user->employee->learningStage; // discovery|routine|mastery|expert
        $alerts = $this->getActiveAlerts($user);
        $modules = $user->company->plan->activeModuleIds();

        return [
            'system' => $this->systemPrompt($user, $stage, $modules),
            'context' => $this->userContext($user, $alerts, $stage),
        ];
    }

    private function systemPrompt(User $u, string $stage, array $mods): string {
        $stoneMap = [
            'discovery' => 'guide bienveillant, phrases courtes, une action à la fois',
            'routine' => 'assistant efficace, réponses directes',
            'mastery' => 'expert pair, propose des optimisations',
            'expert' => 'ressource technique, répond aux questions avancées',
        ];
        return <<<PROMPT
            Tu es Leo, l'assistant IA de Leopardo RH.
            Ton rôle : {$stoneMap[$stage]}.
            Domaines : RH, Finance, Sécurité, utilisation de l'app.
            Modules actifs pour cet utilisateur : {implode(', ', $mods)}.
            Stade d'apprentissage : {$stage}.
            RÈGLE ABSOLUE : Hors de ces domaines → refus poli.
            PROMPT;
        }
    }
}
```

L'onboarding checklist — composant mobile

Pour les utilisateurs en stade 'discovery' (J0-J2), une checklist flottante encourage les 5 premières actions fondatrices. Elle disparaît une fois complétée et ne revient jamais.

Éta pe	Action	Module	Message Leo associé
1/5	Compléter votre profil	Core	Votre profil est complet ! Les autres membres de votre équipe pourront vous identifier.
2/5	Ajouter un premier employé (managers)	RH	Excellent ! Vous pouvez maintenant suivre ses pointages depuis l'onglet RH.
3/5	Faire un premier pointage (employés)	RH	Parfait ! Votre arrivée est enregistrée. Vous pouvez aussi voir votre historique.
4/5	Explorer le module principal de votre plan	Finance ou RH	Ce module vous permet de [action principale]. Posez-moi une question si besoin.

Éta pe	Action	Module	Message Leo associé
5/5	Poser une question à Leo	Leo	Vous savez maintenant utiliser l'essentiel de Leopardo RH. Je suis là si vous avez des questions.

CH.05

HOME SCREEN — SPEC FINALE

Mobile + Back-office web · États · Rôles · Composants

05

Mobile — anatomie zone par zone

Zone	Hauteur	Contenu	Comportement
AppBar	56dp	Avatar (→ ProfileScreen) + 'Bonjour [Prénom]' + icône cloche (badge rouge si alertes non lues)	Sticky. L'avatar génère ses initiales depuis le prénom/nom avec couleur déterministe (hash).
Alertes	0 à 3 × AlertBanner	AlertBanner : icône + message + lien deep. Types : urgent (rouge) / warning (ambre) / info (vert). Dismissible par swipe.	AnimatedSize : 0 si pas d'alerte. Max 3 alertes simultanées. Au-delà → 'Voir toutes les alertes'.
Zone chat (Flex)	Tout l'espace restant	ChatListView : messages user (droite, violet 10%) + Leo (gauche, Slate 50) + TypingIndicator. Scroll to bottom auto.	Skeleton lors du chargement initial. Pull-to-load pour l'historique. Persist dans Hive entre sessions.
ChatInput Bar	56dp (expand si multi-lignes)	TextField Inter + bouton envoi violet. Bordure violet actif / Slate 200 au repos.	Bouton désactivé si champ vide. Haptic feedback à l'envoi. S'expand jusqu'à 4 lignes.
QuickActionsRow	52dp	Row horizontal scrollable. Max 4 chips directs + chip 'Plus →' si > 4.	Couleur chip = domain color du module. Tap → navigation directe. Long press → label visible.
SafeArea bottom	Variable	Respect de la home bar iOS et de la barre de nav Android.	Adaptatif MediaQuery.of(context).padding.bottom.

Web back-office — layout deux colonnes

Zone	Largeur	Contenu	Différence vs mobile
Sidebar gauche	240px fixe	Logo + navigation modules (config-driven) + profil en bas. Couleur fond INK.	Remplace BottomNav. Toujours visible. Icône + label (pas juste icône).
Panneau Leo	320px (collapsible)	Chat Leo complet : messages + input + chips modules. Toggle via bouton.	Panneau latéral persistant sur grand écran. Modal sur < 1280px.
Zone contenu principale	Flex (reste)	Écran du module actif. Si rien de sélectionné : dashboard résumé ou welcome.	Plus dense (tableaux, plus de colonnes). Raccourcis clavier ■K/■N.
Header contenu	56px	Fil d'Ariane + titre écran + actions (boutons, filtres, export).	Remplace l'AppBar mobile. Breadcrumb pour la navigation hiérarchique.
AlertBanner web	Auto-height	Bandeau horizontal en haut de la zone contenu. Max 3. Dismissible.	Position différente du mobile mais même contenu et même logique.

Machine d'états de la Home — complète

```
HomeBloc — machine d'états complète
// États de HomeBloc
sealed class HomeState {
    // Chargement initial (skeleton affiché)
    class HomeLoading extends HomeState {}

    // État principal
    class HomeLoaded extends HomeState {
        final List<HomeAlert> alerts;           // 0 à 3 alertes actives
        final List<ChatMessage> messages;       // Historique messages Leo
        final LearningStage stage;              // Stade d'apprentissage
        final List<ModuleChip> visibleChips;     // Max 4 chips + 'Plus' si needed
        final bool showOnboardingChecklist;     // true si stage=discovery
        final bool hasUnreadAlerts;             // Pour le badge sur la cloche
    }

    // En train d'envoyer un message à Leo
    class HomeSending extends HomeState {
        final List<ChatMessage> messages; // + message user + TypingIndicator
    }

    // Quota journalier atteint
    class HomeQuotaReached extends HomeState {
        final int resetAt; // timestamp du reset (minuit)
    }

    // Mode hors-ligne
    class HomeOffline extends HomeState {
        final List<ChatMessage> cachedMessages; // Messages du cache Hive
        final List<ModuleChip> visibleChips;     // Chips toujours visibles
    }
}
```

Personnalisation par rôle — tableau complet

Élément	Manager	Gestionnaire Finance	Manager RH	Employé simple
Chips (ordre)	RH → Finance → Cam → Rapports → +	Finance → Dépenses → Factures → Achats → +	RH → Absences → Pointage → Rapports → +	Pointer → Congés → Paie → Historique
Alertes visibles	Toutes (RH + Finance + Cam)	Finance uniquement	RH uniquement	Ses propres alertes uniquement
Morning brief	Oui : résumé global	Oui : résumé Finance	Oui : résumé RH	Non : Leo attend la question
Onboarding steps	5 étapes (profil + RH + Finance)	4 étapes (profil + Finance)	4 étapes (profil + RH)	2 étapes (profil + pointer)
Leo scope	Tout Leopardo	Finance + App (0 données RH)	RH + App (0 Finance)	Ses données uniquement

Élément	Manager	Gestionnaire Finance	Manager RH	Employé simple
Raccourci web ■■N	Employé / Facture / Dépense	Facture / Dépense / BC	Employé / Absence / Congé	N/A (employé = mobile only)
Back-office web	Oui — accès complet	Oui — modules Finance seulement	Oui — modules RH seulement	Non — mobile uniquement

CH.06

LES 48 TÂCHES

4 sprints · Maintenant vs Phase 2 clairement séparés

06

Deux types de tâches : MAINTENANT (pose les fondations, active les features) et PHASE 2 (marquées [P2], à ne pas implémenter maintenant mais à ne pas oublier). Les tâches MAINTENANT sont prioritaires et bloquantes pour l'architecture cible.

SPRINT 1 —
Architecture core

Packages Flutter · Contrat module · Route groups Laravel · ~6 j-dev

- T-APV-01** **Package leopardo_core**
Créer le package séparé depuis lib/core/. Migrer : AppColors v3, AppTheme (light+dark), ModuleRegistry, AppShell, LeopardoBadge, EmptyState, AlertBanner, ErrorMessage. Tester avec flutter pub get isolé. 1 j · Flutter
- T-APV-02** **LeopardoModule contract**
Créer abstract class LeopardoModule dans leopardo_core (voir Ch.03). id, label, icon, domainColor, minPlan, routes, bottomNavBar, getAlerts(), leoTopics, homeChip, onboardingSteps. README.md. 0.5 j · Flutter
- T-APV-03** **ModuleRegistry v2**
Mettre à jour utiliser LeopardoModule. registerAll(). tabsForPlan(). alertsForUser(). chipsForHome(max:4). leoTopicsForUser(). onboardingStepsForUser(). Plus aucun hardcode. 0.5 j · Flutter
- T-APV-04** **Packages modules (rh, finance, cameras)**
Créer 3 packages. Migrer le code existant. Chaque module implémente LeopardoModule. Vérifier que chaque package compile seul. CI job 'package-isolation'. 1 j · Flutter
- T-APV-05** **Route groups Laravel isolés**
Créer routes/modules/m.php, finance.php, cameras.php. Inclure dans api.php. Vérifier isolation des middlewares. Tests Pest : route d'un module non accessible sans son middleware. 0.5 j · Web Blade
- T-APV-06** **config/modules.php Laravel**
Source de vente des modules Web. Synchronisé avec ModuleRegistry Flutter. Sidebar générée depuis ce tableau. Chaque module : id, label, icon, min_plan, color, routes_prefix, sidebar_label. 0.25 j · Web Blade
- T-APV-07** **metadata JSONB sur tables clés**
Migration : employees.metadata (app_actions_count, learning_stage, first_seen_screens, preferred_chips_order). companies.metadata. Invoice et expense déjà dans les specs modules. 0.5 j · Web Blade
- T-APV-08** **Feature flags system**
Feature flag generated (feature', \$user) → bool. Stocké dans companies.features JSONB. Toggle super-admin. Retourné dans /api/v1/auth/me.features. Utilisé par les modules pour les sub-features. 0.5 j · Web Blade
- T-APV-09** **Tests isolation packages (CI)**
Créer Actions job flutter pub get + flutter analyze dans chaque package séparément. Vérifier que 'import module_finance' packages/module_rh = 0 résultat. Pest : routes modules isolées. 0.5 j · Transversal

T-APV-10

AppColors v3 + design tokens synchronisés

AppColors dans final (toutes couleurs domaines), Tailwind.config.js synchronisé. STATUTS.md créé. Transversal COULEURS.md créé. CI job qui compare les valeurs hex entre les deux fichiers.

SPRINT 2 — Leo + Onboarding intégrés

LeoContextService · HomeBloc v2 · Checklist · Morning brief · ~9 j-dev

- T-APV-11

LeoContextService (Laravel)

Service central qui construit le prompt système complet avec : rôle, plan, stade learning, alertes actives, modules actifs, écrans vus. Support multi-provider (OpenAI/Anthropic). buildPrompt(user, mode).

1.5 j · Web Blade
- T-APV-12

LearningStageService + ActionTracker

Service qui calcule le stade depuis metadata.app_actions_count. ActionTrackerMiddleware : incrémente après chaque POST/PATCH/DELETE réussie. Batch update tous les 5 actions (pas de requête par action).

0.75 j · Web Blade
- T-APV-13

POST /api/v1/ai/chat

Endpoint chat. Vérifie quota (MessageQuotaMiddleware). Appelle LeoContextService.buildPrompt(). Appelle provider IA. Log usage. Retourne message + suggestions contextuelles. Throttle 30/min.

0.75 j · Web Blade
- T-APV-14

GET /api/v1/ai/morning-brief

Retourne le brief matinal. C'est la première ouverture du jour ET alertes actives. Inclut le stade learning pour adapter le ton. Mis en cache 1h (évite les appels redondants).

0.5 j · Web Blade
- T-APV-15

GET /api/v1/ai/suggestions

Retourne 2-3 suggestions contextuelles selon le rôle, l'heure, le stade et les alertes. Permet d'afficher des suggestions pré-écrites sous la barre chat. Cache 15min.

0.5 j · Web Blade
- T-APV-16

HomeConversationalScreen Flutter

Ecran complet selon Ch.05. AppBar + AlertBanners + ChatListView + ChatInputBar + QuickActionsRow + SafeArea. Tous les états de HomeBloc couverts. Skeleton pendant loading.

1.5 j · Flutter
- T-APV-17

HomeBloc v2 complet

Tous les états (HomeLoading, HomeLoaded, HomeSending, HomeQuotaReached, HomeOffline). Events: LoadHome, SendMessage, DismissAlert, TapQuickAction. Lit le learning_stage depuis auth/me.

1 j · Flutter
- T-APV-18

Onboarding checklist Flutter

Composant OnboardingChecklist visible si stage=discovery. 5 étapes configurées depuis LeopardoModule.onboardingSteps. Dismiss une fois toutes les étapes complétées. Progress bar. Persisté dans metadata.

0.75 j · Flutter
- T-APV-19

Progressive disclosure Flutter

LearningStageProvider (Riverpod). shouldReveal(feature, stage) → bool. Wraps les features avancées (filtres, exports, config). Lu depuis auth/me.learning_stage. Hot-reload sans redémarrage.

0.75 j · Flutter
- T-APV-20

Home back-office web (Blade)

Layout 2 colonnes + panneau Leo (sidebar gauche + Leo + contenu principal). x-leo-panel, x-module-chips, x-alert-banner. Responsive : Leo devient modal < 1280px. Progressive disclosure via @whenStage().

1 j · Web Blade

SPRINT 3 —
Mobile-first & polish

Review process · Offline · Empty states · Accessibilité · ~7 j-dev

T-APV-21

Process review iPhone SE obligatoire

Documenter dans CONTRIBUTING.md : toute PR UI doit inclure un screenshot simulateur iPhone SE (375px). CI job optionnel golden_toolkit. Réviser les issues existantes identifiées à 375px.

0.25 j · Transversal

T-APV-22

Mode hors-ligne gracieux Flutter

connectivity_plus : offline : banner discret 'Hors ligne'. Cache Hive : derniers messages Leo + listes modules (employés, factures). Quick Actions visibles mais avec icône offline. Sync auto au retour.

0.75 j · Flutter

T-APV-23

Empty states intelligents (Leo-linked)

Chaque empty state propose d'interagir avec Leo. EmptyState widget : illustration + titre + message Leo + CTA button. Ex: 'Posez-moi 'créer une facture' pour commencer'. Deep link vers la barre chat.

0.5 j · Transversal

T-APV-24

Raccourcis clavier web (⌘K, ⌘N)

Appeler global listener. ⌘K → focus barre Leo. ⌘N → modal quick-create (items selon modules actifs). Escape → ferme. Visible uniquement au stade 'mastery' et au-delà (progressive disclosure).

0.5 j · Web Blade

T-APV-25

Accessibilité WCAG AA

Flutter : Semantics sur tous les éléments interactifs. Contraste 4.5:1 minimum vérifié. Web : axe-core dans CI. Tous les inputs avec labels associés. ChatInputBar : aria-label='Poser une question à Leo'.

0.75 j · Transversal

T-APV-26

Cohérence cross-surface validée

Créer le test de cohérence : LeopardoBadge flutter = x-attendance-badge blade (mêmes hex). Automatiser dans CI. STATUTS.md mis à jour. Réviser toutes les couleurs statuts existantes.

0.5 j · Transversal

T-APV-27

18n architecture (FR → arabe RTL ready)

APB files par module dans packages/module_rh/lib/l10n/. Fichier lang/ par module Laravel. Structure RTL-ready : pas de hardcoded left/right → start/end (Flutter Directionality). Leo répond dans la langue de l'interface.

0.5 j · Transversal

T-APV-28

Performance baseline

Flutter analyze → 0 warnings. Mesurer : APK size avant/après packages isolés. Lighthouse web > 90. EXPLAIN ANALYZE sur /api/v1/ai/morning-brief et /api/v1/auth/me. Documenter les baselines dans /docs/PERFORMANCE.md.

0.5 j · Transversal

T-APV-29

Notification mobile intelligente

Max 3 notif/jour par utilisateur. Groupées par module. Deep link vers l'entité. Respect silence nocturne (22h-8h local). Leo ne notifie jamais si l'app est ouverte (message in-app à la place).

0.5 j · Flutter

T-APV-30

Documentation in-app via Leo

Chaque écran : bouton '?' discret → ouvre Leo avec 'Explique-moi cet écran [screenId]'. LeoContextService retourne une explication adaptée au stade de l'utilisateur. Pas de PDF de documentation.

0.5 j · Transversal

SPRINT 4 —
Fondations Phase 2
(sans l'activer)

Contrats stables · CI isolation · Guide contribution · [P2] marquées · ~8 j-dev

● T-APV-31	API versioning ready Créer routes/api_v2.php vide mais structuré. ApiVersionMiddleware (header Accept-Version). Documenter la procédure de migration V1→V2 dans /docs/API_VERSIONING.md. Chaque endpoint a son numéro de version.	0.5 j · Web Blade
● T-APV-32	Contrats API stables documentés Mettre à jour 02_API_CONTRATS_COMPLET.md avec tous les endpoints APV (chat, morning_brief, suggestions, auth/me avec learning_stage). Chaque endpoint a son schema OpenAPI complet.	0.5 j · Web Blade
● T-APV-33	[P2] Dart deferred imports — structure Ajouter un commentaire // TODO Phase 2 : deferred as rh dans main.dart aux endroits précis. Ne pas activer maintenant. Documenter dans CONTRIBUTING_MODULE.md la procédure pour Phase 2.	0.25 j · Flutter
● T-APV-34	[P2] Shorebird config — prêt pas activé Créer shorebird.yaml avec la config correcte. Ne pas l'intégrer dans le CI actuel. Documenter dans /docs/ShOREBIRD.md la procédure d'activation Phase 2 et les contraintes iOS (pas de code natif).	0.25 j · Flutter
● T-APV-35	Guide contribution module (CONTRIBUTING_MODULE.md) Guide pas à pas : (1) créer le package, (2) implémenter LeopardModule, (3) déclarer les routes Laravel, (4) configurer config/modules.php, (5) checklist PR. Template package Flutter inclus.	0.75 j · Transversal
● T-APV-36	APV.md — document vivant Créer /docs/APV.md résumé d'une page du manifeste. Les 3 décisions. Les 12 Lois. Le contrat module. Consulté à chaque PR. Mis à jour si une Loi change. Lien dans PILOTAGE.md et README.md.	0.5 j · Transversal
● T-APV-37	Métriques d'usage anonymes Compter sans logs les plus visités, questions Leo les plus posées, modules les plus utilisés, taux d'adoption par stade. Dashboard super-admin. Guide les décisions produit Phase 2.	0.5 j · Web Blade
● T-APV-38	Changelog in-app par module Modal 'Nouveautés' affiché une fois par version. Filtre : montre uniquement les features des modules actifs de l'utilisateur. Lien 'Demander à Leo' pour chaque feature. Versionné via metadata.last_seen_version.	0.5 j · Flutter
● T-APV-39	Regression tests visuels Golden Toolkit Flutter : HomeScreen, LoginScreen, AttendanceScreen en light+dark. Dusk Laravel : home web, employees index. Screenshots commitées. CI compare à chaque PR. Diff > 1% → PR bloquée.	0.75 j · Transversal
● T-APV-40	Monitoring erreurs (Sentry) Flutter : Sentry SDK. Capture erreurs non-gérées avec contexte (module actif, stade learning, état Bloc). Web : Sentry JS. Alerting si taux d'erreur > 1%. Super-admin voit les erreurs groupées par module.	0.5 j · Transversal
● T-APV-41	Load testing complet K6 100 users simultanées sur /api/v1/ai/chat. Objectif p95 < 800ms. 50 users sur /api/v1/auth/me. Objectif p95 < 200ms. Documenter dans PERFORMANCE.md. Alerting si dégradation > 20%.	0.5 j · Transversal
● T-APV-42	Mettre à jour PILOTAGE.md Ajouter APV comme section 'Architecture vivante'. Mettre à jour PROGRAM_VERSION. Ajouter les nouvelles routes APV dans la liste des endpoints. Retirer les features APV de la liste EXCLUDED.	0.25 j · Transversal
● T-APV-43	STATUTS.md final et CI check STATUTS.md complet avec toutes les couleurs de tous les statuts sur les deux surfaces. Script Python CI qui extrait les hex de AppColors.dart et tailwind.config.js et les compare. 0 divergence tolérée.	0.25 j · Transversal

- T-APV-44

Accessibilité mobile terrain (test réel)

Tester VoiceOver iOS sur HomeScreen + ChatInputBar. Tester TalkBack Android sur LeopardoBadge+. QuickActionsRow. Corriger les Semantics manquants. Documenter les résultats dans /docs/ACCESSIBILITY.md.

0,5 j · Flutter
- T-APV-45

IP2] Module Federation web — structure

Ajouter webpack.config.js exemple avec la config Module Federation commentée. Ne pas activer. Documenter le chemin de migration Phase 2 dans /docs/MICROFRONTENDS.md.

0,25 j · Web Blade
- T-APV-46

IP2] Personnalisation chips (drag-and-drop)

ICD commenté dans QuickActionsRow : ordre des chips sauvegardé dans metadata.preferred_chips_order. Implémentation Phase 2. L'ordre actuel = défaut du rôle.

0,25 j · Flutter
- T-APV-47

IP2] Leo suggestions proactives (ML)

ICD commenté dans LeoContextService : analyser l'historique des 30 derniers jours pour personnaliser les suggestions. Phase 3. Phase 2 = suggestions règles-based depuis l'historique récent.

0,25 j · Web Blade
- T-APV-48

IP2] Portail onboarding client SaaS

ICD : page web publique d'onboarding pour les nouveaux clients Enterprise (sans login). Guide vidéo + checklist + contact commercial. Phase 2. Pour l'instant : onboarding in-app via Leo suffit.

0,25 j · Transversal

Sprint	Thème	Tâches	Effort	Livrable
Sprint 1	Architecture core	10	~6 j-dev	Packages isolés · Contrat module · Route groups
Sprint 2	Leo + Onboarding	10	~9 j-dev	Home conversationnelle · LeoContextService · Checklist
Sprint 3	Mobile-first & polish	10	~6,5 j-dev	Process review · Offline · Empty states · A11y
Sprint 4	Fondations Phase 2	18	~7 j-dev	API versioning · Guides · Monitoring · [P2] documentées
TOTAL	Architecture Produit Vivante v2	48 tâches	~28,5 j-dev	Fondations solides pour Phase 2 micro-frontends

LES 12 LOIS

Immuables · La source de vérité absolue · Valent pour toujours

07

Ces 12 Lois ne sont pas des recommandations. Ce sont des contraintes architecturales immuables. Modifier une Loi requiert : (1) justification écrite, (2) accord équipe produit, (3) mise à jour de ce document ET de /docs/APV.md simultanément. Une PR qui viole une Loi sans l'avoir modifiée est refusée automatiquement.

L01

Home = Conversation

Le premier écran après login est la Home conversationnelle avec Leo. Sur les deux surfaces. Le dashboard classique existe en tant que destination — jamais comme point d'entrée. Cette loi est immuable.

L02

Mobile est la source de vérité UX

Toute spec UX commence par l'écran mobile. Le web adapte. Jamais l'inverse. Toute review design se fait d'abord sur simulateur iPhone SE (375px). Une feature conçue web-first est une violation.

L03

Web = back-office managers exclusivement

L'employé simple n'a pas accès au web et n'en a pas besoin. Le web est conçu pour 1024px minimum, layout deux colonnes, usage clavier. Pas de responsive mobile côté web.

L04

Core n'importe aucun module

leopardo_core ne contient aucune référence à Finance, Caméras, RH, ou tout autre module. Un import direct d'un module dans le core est une violation critique. Le CI vérifie automatiquement.

L05

Modules ne se connaissent pas entre eux

module_finance ne peut pas importer module_cameras. Communication uniquement via événements publiés sur le bus core. Un import direct inter-modules est une violation critique. Le CI vérifie.

L06

Couleur = domaine, toujours

Vert = RH. Ambre = Finance. Bleu = Sécurité. Violet = IA/Chat. Rouge = Erreur. Ces associations sont permanentes. Les deux surfaces doivent être synchronisées. STATUTS.md fait foi.

L07

Leo est spécialisé et guidé par le stade

Leo répond uniquement aux questions Leopardo RH. Le system prompt inclut toujours le stade learning et le ton adapté. Modifier le scope ou le ton de Leo = modification de LeoContextService documentée.

L08

Onboarding et Leo partagent le même service

LeoContextService est le seul service qui connaît le stade learning. L'onboarding déclenche Leo. Leo sait où en est l'utilisateur. Pas deux systèmes séparés pour apprendre et guider.

L09

Micro-frontends Phase 2, fondations maintenant

On ne déploie pas de modules indépendants maintenant. On pose les contrats, les packages, et la structure. Aucune tâche marquée [P2] ne peut être implémentée avant la validation formelle du passage en Phase 2.

L10**Contrat LeopardoModule est stable par release**

L'interface LeopardoModule ne change pas dans une minor release. Un breaking change = nouvelle major version de leopardo_core. Tous les modules ont 30 jours pour migrer.

L11**JSONB d'abord pour les nouvelles données**

Les nouvelles données expérimentales vont dans le champ metadata JSONB. La migration vers colonne dédiée se fait quand la feature est stabilisée. Pas de migration de schéma pour du code expérimental.

L12**Progressive disclosure avant feature flag**

Avant de cacher une feature derrière un flag, demander : peut-elle être cachée via le stade learning ? Si oui, utiliser le système de stades. Les feature flags sont pour les features non-stables uniquement.

Ce manifeste est la version 2 de l'Architecture Produit Vivante. Il intègre les 3 décisions confirmées : Mobile-first · Micro-frontends Phase 2 · Leo + Onboarding intégrés. Toute modification d'une Loi est trackée dans le git blame de ce fichier.
Version 2.0 — Avril 2026 — Équipe Produit Leopardo RH.